

INDIA.RUNS

Build what next India runs on

Title: Track 01 - The Data & AI Challenge

Team Leader Name: Bhavya Keerthi K

Problem Statement: Build an AI brain for modern hiring that ranks candidates based on actual suitability, production ML experience, and behavioral availability instead of generic keyword matching, under strict CPU-only and local execution limits.

Solution Overview

- **What is your proposed solution?**

A high-performance, deterministic stream ranking engine that processes 100,000 candidate profiles sequentially.

The system parses and combines career summaries, title hierarchies, validated skills, and behavioral platform activity, outputting the top 100 candidates in under 37 seconds on a standard CPU.

- **What differentiates your approach from traditional candidate matching systems?**

Traditional systems rely on simple keyword string matching (like BM25) or make expensive per-candidate Large Language Model (LLM) calls, which fail under scale and time constraints.

Our approach uses a multi-faceted feature extraction scoring formula combined with a strict verification guard layer that immediately identifies and penalizes keyword-stuffing and impossible profiles (honeypots).

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

Experience Sweet Spot: Total experience of 5 to 9 years, with 4+ years of actual hands-on ML/AI experience.

Production Shipper Profile: Deployed embeddings-based retrieval systems and vector databases (FAISS, Milvus, Pinecone) in production for real users.

Evaluation Competency: Designing evaluation frameworks for ranking systems (NDCG, MRR, MAP).

Product Mindset: Exposure to product-focused companies (SaaS, AI/ML, E-commerce) over services-only companies.

- How does your solution evaluate candidate fit beyond keyword matching?

Evaluates title hierarchy rather than keyword presence, down-weighting non-target titles (e.g. Marketing Manager) even if they list AI skills.

Adjusts skill weights using a trust multiplier based on proficiency, duration (months), endorsements, and verified platform assessment scores.

Screens career history for active keywords like "A/B testing", "NDCG", "deployed", "real users", and "evaluation framework" to separate actual builders from demo builders.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

Candidate profiles are streamed one-by-one (low memory).

Each profile is evaluated by independent feature scorers (Title, Career, Skills, Experience, Product, Availability).

Scores are compared, and qualifying profiles are placed into an in-memory min-heap of size 100, sorting candidates best-fit first and breaking score ties deterministically using candidate IDs.

- **What models, algorithms, or heuristics are used?**

Weighted linear scorer: $\text{Score} = 0.24 * \text{TitleScore} + 0.25 * \text{CareerScore} + 0.18 * \text{SkillScore} + 0.10 * \text{ExperienceScore} + 0.10 * \text{ProductScore} + 0.13 * \text{AvailabilityScore}$

Substring company matching to identify product vs. service experience.

Chronological calculations to count years in AI/ML specific roles.

- **How are multiple candidate signals combined into a final ranking?**

Static details (titles, experience, location), semantic descriptions, and dynamic platform behavior (login recency, response speed, notice period) are combined into a single numeric score clamped between 0.0 and 1.0.

A terminal penalty layer reduces the score of suspicious profiles to 0.0.

Explainability & Data Validation

- **How are ranking decisions explained?**

For every candidate in the top 100 shortlist, the engine outputs a 1-2 sentence reasoning text. This text highlights their role title, location, years of experience, specific shipped skills, product exposure, and availability.

- **How do you prevent hallucinations or unsupported justifications?**

The reasoning text is constructed programmatically using the same verified database values and features that computed the score, ensuring that every claim in the explanation is factually grounded and free of LLM hallucinations.

- **How does your solution handle inconsistent, low-quality, or suspicious profiles?**

The guard layer applies a terminal penalty of 1.0 (reducing the final score to 0.0) for:

Founding Date Mismatches: e.g., claiming to work at Krutrim or Sarvam AI before 2023, or Pinecone before 2019.

Zero-Duration Expert Skills: Listing expert or advanced skills with 0 months of use.

Timeline Anomalies: Role duration exceeding overall experience, or dates spanning a different duration than stated.

Job Hopping: Penalizes candidates with average tenures under 18 months who rapidly climb titles across companies.

End-to-End Workflow

JD Input: Read and parse the job description to define the target weights, cities, and company limits.

Stream JSONL: Read candidates sequentially from the dataset file, avoiding loading the 487 MB file into memory.

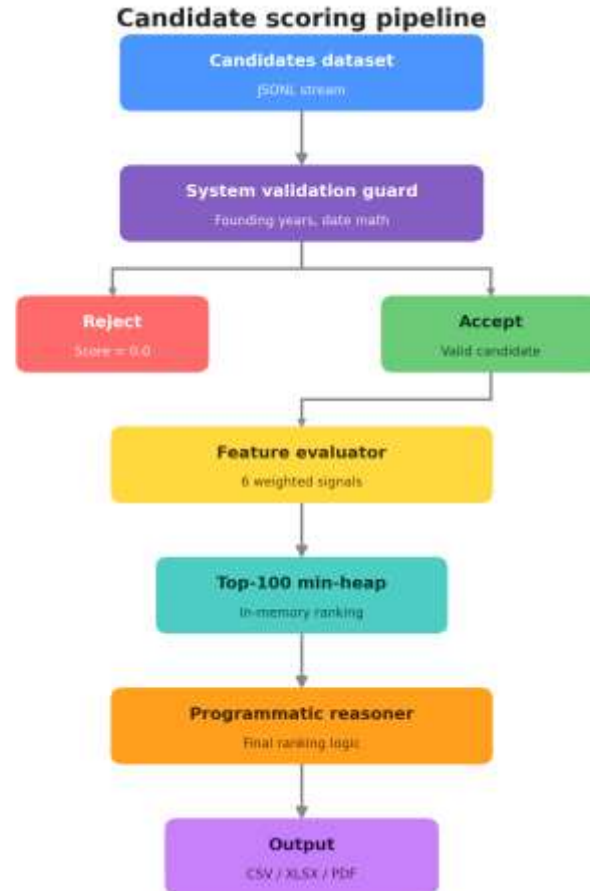
Guard Checks: Run chronological and founding year validations. Immediately flag honeypots.

Compute Score: For valid candidates, calculate scores for title fit, career signals, skills, experience, product background, and availability.

Select Top 100: Maintain the top candidates in a min-heap structure.

Output Shortlist: Export the ranked candidates with programmatically generated reasoning to CSV and XLSX.

System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

Honeypot Rate: 0% in the top 100 (all impossible profiles are filtered out).

shortlists: The top 3 ranked candidates are matching senior profiles with relevant vector search and recommendation systems experience.

- How does your solution meet the challenge's runtime and compute constraints?

Runtime: ~37 seconds to stream, score, and rank 100,000 candidates (limit: 5 minutes).

Memory: Uses less than 15 MB of RAM (limit: 16 GB).

Hardware: Runs entirely locally on CPU with zero network dependencies (free of API timeouts or rate limits).

Technologies Used

Python 3.11 (Standard Library): Used as the core programming language. Built entirely with standard library imports (gzip, json, csv, math, heapq) to ensure zero setup dependencies and high-speed execution.

Pandas & Openpyxl: Selected for fast, clean generation of the final XLSX spreadsheet.

ReportLab: Selected to build the presentation PDF dynamically from code, ensuring precise control over layout without formatting drift.

Git: Used for tracking changes, branch management, and providing a clean repository history.

Docker: Configured to test the application within a clean, sandboxed container.

Submission Assets

GitHub Repository URL: <https://github.com/bhavyakeerthi3/redrob-ai-candidate-ranker>

Submission Spreadsheets:

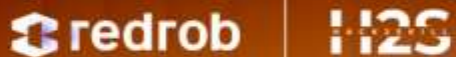
CSV format: outputs/team_redrob_ranker.csv

XLSX format: outputs/team_redrob_ranker.xlsx

Presentation Decks:

PDF format: outputs/redrob_candidate_ranker_deck.pdf

PPTX format: outputs/redrob_candidate_ranker_deck.pptx



INDIA.RUNS

Build what next India runs on

THANK YOU